

Day 15 Pandas Part 1

Gemini said

Pandas Series

A Pandas Series is a one-dimensional labeled array designed to hold data along with its index.

```
import pandas as pd

# Creating a Series for a single variable (e.g., student scores)
scores = pd.Series([85, 92, 78, 90], index=['Alice', 'Bob', 'Charlie',
'David'])
print(scores)
```

Pandas DataFrame & Data Input

A DataFrame is a two-dimensional labeled data structure where each column can contain different data types. Pandas can seamlessly read data from multiple sources like CSV, Excel, and JSON.

+1

```
# Creating a DataFrame from a dictionary to handle multiple variables
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [24, 27, 22, 32],
    'City': ['NY', 'LA', 'Chicago', 'Houston']
}
df = pd.DataFrame(data)

# Example of reading real-world data from a CSV file
# df_csv = pd.read_csv('historical_stock_prices.csv')

print(df)
```

Selection and Indexing

You can access specific rows using `.loc` for label-based indexing or `.iloc` for integer positions. Conditional selection allows you to filter data based on specific criteria, and setting an index helps with easy lookups.

+2

```
# Accessing rows using .iloc (integer position)
print("First row:\n", df.iloc[0])

# Conditional Selection
adults = df[df['Age'] > 25]
print("\nPeople over 25:\n", adults)

# Setting a column as the index
df_indexed = df.set_index('Name')
print("\nLookup by index:\n", df_indexed.loc['Bob'])
```

Operations on DataFrames

Pandas provides numerous methods to inspect and manipulate data, such as viewing top/bottom rows, finding unique values, sorting columns, and applying custom functions.

+3

```
# Inspecting data
print("First 2 rows:\n", df.head(2))
print("Last 2 rows:\n", df.tail(2))

# Understanding data
print("\nUnique cities:", df['City'].unique())
print("City value counts:\n", df['City'].value_counts())

# Sorting and custom functions
df_sorted = df.sort_values(by='Age')
df['Age_in_5_years'] = df['Age'].apply(lambda x: x + 5)
print("\nUpdated DataFrame:\n", df)
```

Missing Data Handling

Missing or null values can break calculations. You can handle this by identifying them, dropping the affected rows/columns, or filling them with substitute values like the mean or using forward/backward fill techniques.

+2

```
import numpy as np

# Creating a DataFrame with missing values (NaN)
df_missing = pd.DataFrame({
    'Revenue': [1000, 2000, np.nan, 4000],
    'Units': [50, np.nan, np.nan, 80]
})

# Check for null values
print("Checking for nulls:\n", df_missing.isnull())

# 1. Drop rows with missing values
df_dropped = df_missing.dropna()

# 2. Fill missing values with the mean
df_filled_mean = df_missing.fillna(df_missing.mean())

# 3. Forward fill (replace missing value with the previous value in the column)
df_ffill = df_missing.ffill()

print("\nForward Filled Data:\n", df_ffill)
```

Would you like to try applying these concepts to a sample dataset to see how they work together?